

# CS 8803 HML - Lab 3

## Screenshots

```
[kmadaswar3@atl11-1-03-017-23-0 lab3]$ make test
pytest -q
.....
]
10 passed in 67.52s (0:01:07)
```

Figure 1: Pytest Results

```
[kmadaswar3@atl11-1-03-017-23-0 lab3]$ make bench
python scripts/bench_gemm.py --dtype fp16 --device cuda --m 1024 --n 1024 --k 1024
workload=gemm+bias+relu shape=1024x1024x1024 dtype=fp16 device=cuda relu=True
torch.linear 100.03 TFLOP/s (0.021 ms)
fused_linear 74.10 TFLOP/s (0.029 ms)
tiled_gemm+bias+relu 51.66 TFLOP/s (0.042 ms)
naive_gemm+bias+relu 4.87 TFLOP/s (0.441 ms)
python scripts/bench_gemm.py --dtype fp16 --device cuda --m 2048 --n 2048 --k 2048
workload=gemm+bias+relu shape=2048x2048x2048 dtype=fp16 device=cuda relu=True
torch.linear 560.84 TFLOP/s (0.031 ms)
fused_linear 329.31 TFLOP/s (0.052 ms)
tiled_gemm+bias+relu 188.47 TFLOP/s (0.091 ms)
naive_gemm+bias+relu 5.26 TFLOP/s (3.268 ms)
python scripts/bench_gemm.py --dtype fp16 --device cuda --m 4096 --n 4096 --k 4096
workload=gemm+bias+relu shape=4096x4096x4096 dtype=fp16 device=cuda relu=True
torch.linear 682.01 TFLOP/s (0.202 ms)
fused_linear 343.02 TFLOP/s (0.401 ms)
tiled_gemm+bias+relu 197.60 TFLOP/s (0.696 ms)
naive_gemm+bias+relu 5.36 TFLOP/s (25.648 ms)
```

Figure 2: Benchmark Results

```
(hml) [kmadaswar3@atl11-1-03-017-9-0 scratch]$ make demo
python scripts/run_mlp_demo.py --dtype fp16 --compile 0
mode=eager dtype=fp16 batch=256
torch_mlp: 0.183 ms
unfused_mlp: 0.176 ms
fused_mlp: 0.153 ms
fused_vs_unfused: 1.151x
fused_vs_torch: 1.193x
unfused_vs_torch: 1.037x
```

Figure 3: MLP Demo Results

# Kernel Design

## 1. Naive GEMM (`gemm_naive.py`)

The naive kernel computes each output element by iterating over the shared K dimension one scalar at a time, multiplying the corresponding elements of A and B and accumulating the result. It never uses tensor cores, so performance is bottlenecked by scalar arithmetic and tops out around 5 TFLOP/s regardless of matrix size.

## 2. Tiled GEMM (`gemm_tiled.py`)

Instead of working scalar by scalar, the tiled kernel loads rectangular blocks of A and B into registers and uses `tl.dot()` to multiply them. Unlike the naive per-element multiply-accumulate, `tl.dot()` issues a single warp-level matrix multiply instruction that dispatches to the GPU’s tensor cores, which can compute thousands of FMAs in one instruction. This single change improves throughput by 40× over the naive kernel.

## 3. Fused GEMM (`gemm_fused.py`)

Builds on the tiled kernel by adding the bias addition and ReLU activation into the same kernel rather than launching them as separate operations. Since the result of the matrix multiply is already in registers, applying bias and ReLU there avoids writing intermediate results to global memory just to read them back. This gives a speedup of 1.16× over the unfused path and 1.12× over PyTorch’s `nn.Linear`.

# Profiling Analysis

Profiled on a  $4096 \times 4096 \times 4096$  fp16 GEMM (NVIDIA H200).

| Metric                | Naive      | Tiled       |
|-----------------------|------------|-------------|
| Duration              | 33.70 ms   | 0.78 ms     |
| Compute Throughput    | 89.63%     | 24.56%      |
| Memory Throughput     | 80.52%     | 91.84%      |
| Memory BW (effective) | 23.11 GB/s | 253.98 GB/s |
| Achieved Occupancy    | 98.72%     | 48.47%      |

Table 1: Nsight Compute metrics for naive and tiled GEMM kernels.

**Naive kernel:** High compute throughput (89.6%) and occupancy (98.7%) are misleading as the kernel is busy doing scalar FMAs rather than tensor core MACs. Despite high utilization, effective memory bandwidth is only 23 GB/s because loads are serialized one element at a time across K. The result is a 33.7 ms runtime for a problem that the tiled kernel solves in under 1 ms.

**Tiled kernel:** Memory throughput jumps to 91.8% with 254 GB/s effective bandwidth, while compute throughput drops to 24.6%, showing the kernel is now memory-bound. `tl.dot()` dispatches to tensor cores which are fast enough to consume each tile before the next one arrives from memory, leaving the SM stalled waiting on loads rather than doing compute. Occupancy falls to 48.5% because holding the large accumulator tile in registers across the K loop requires 64 registers per thread (vs. 32 for naive), which halves the number of warps that can fit on the SM simultaneously.

## Optimization Attempts and Improvement Ideas

1. **Software pipelining (num\_stages):** Increasing `num_stages` in the tiled kernel lets Triton overlap global-memory loads for the next tile with the `tl.dot()` compute for the current tile. Increasing `num_stages` from 1 to 2 improved throughput from 218 to 331 TFLOP/s, but returns diminish beyond that because each additional stage consumes more shared memory, reducing the number of concurrent blocks per SM and lowering occupancy.
2. **Block size tuning:** Increasing `BLOCK_M=BLOCK_N` from 64 to 128 and `BLOCK_K` from 32 to 64 improves tensor-core utilization by performing more useful work per block, speeding up the kernel from 342 to 454 TFLOP/s on shape  $4096 \times 4096 \times 4096$ . The benefit is not visible for smaller shapes like  $1024 \times 1024 \times 1024$  because larger tiles produce fewer total blocks ( $8 \times 8 = 64$  vs  $16 \times 16 = 256$ ), leaving most of the 132 SMs idle.